Integrations and observability

Attach MCP-backed capabilities and inspect runs before you start tuning.

Copy Page

After the workflow shape is clear, the next questions are which external surfaces should live inside the agent loop and how you will inspect what actually happened at runtime.

Choose what lives in the SDK

Need	Start with	Why
Give an agent access to public, remotely hosted MCP tools	Hosted MCP tools in the SDK	The model can call the remote MCP server through the hosted surface
Connect local or private MCP servers from your runtime	SDK-managed MCP servers over stdio or streamable HTTP	Your runtime owns the connection, approvals, and network boundaries
Debug prompts, tools, handoffs, or approvals	Built-in tracing	Traces show the end-to-end record before you formalize evals

Tool capability semantics still live in Using tools. This page focuses on the SDK-specific MCP wiring and observability loop.

MCP

Use hosted MCP tools when the remote server should run through the model surface.

Attach a hosted MCP server — typescript

```typescript
import { Agent, hostedMcpTool } from "@openai/agents";

const agent = new Agent({
  name: "MCP assistant",
  instructions: "Use the MCP tools to answer questions.",
  tools: [
    hostedMcpTool({
      serverLabel: "gitmcp",
      serverUrl: "https://gitmcp.io/openai/codex",
    }),
  ],
});
```

Use local transports when your application should connect to the MCP server directly.

Connect a local MCP server — typescript

```typescript
import { Agent, MCPServerStdio, run } from "@openai/agents";

const server = new MCPServerStdio({
  name: "Filesystem MCP Server",
  fullCommand: "npx -y @modelcontextprotocol/server-filesystem ./sample_files",
});

await server.connect();

try {
  const agent = new Agent({
    name: "Filesystem assistant",
    instructions: "Read files with the MCP tools before answering.",
    mcpServers: [server],
  });

  const result = await run(agent, "Read the files and list them.");
  console.log(result.finalOutput);
} finally {
  await server.close();
}
```

The practical split is:

- Use **hosted MCP** for public remote servers that fit the platform trust model.
- Use **local or private MCP** when your runtime should own connectivity, filtering, or approvals.

For the platform-wide concept, trust model, and product support story, keep MCP and Connectors as the canonical reference.

Tracing

Tracing is built into the Agents SDK and is enabled by default in the normal server-side SDK path. Every run can emit a structured record of model calls, tool calls, handoffs, guardrails, and custom spans, which you can inspect in the Traces dashboard.

The default trace usually gives you:

Copy Page

- the overall run or workflow
- each model call
- tool calls and their outputs
- handoffs and guardrails
- any custom spans you wrap around the workflow

If you need less tracing, use the SDK-level or per-run tracing controls rather than removing all observability from the workflow.

Wrap multiple runs in one trace

typescript

```typescript
import { Agent, run, withTrace } from "@openai/agents";

const agent = new Agent({
  name: "Joke generator",
  instructions: "Tell funny jokes.",
});

await withTrace("Joke workflow", async () => {
  const first = await run(agent, "Tell me a joke");
  const second = await run(agent, `Rate this joke: ${first.finalOutput}`);
  console.log(first.finalOutput);
  console.log(second.finalOutput);
});
```

Use traces for two jobs:

- Debug one workflow run and understand what happened.
- Feed higher-signal examples into agent workflow evaluation once you are ready to score behavior systematically.

Next steps

Once the external surfaces are wired in, continue with the guide that covers capability design, review boundaries, or evaluation.

Using tools
See how hosted tools, function tools, and agents-as-tools fit beside MCP.

Guardrails and human review
Add approval or validation boundaries around sensitive capabilities.

Agent workflow evaluation
Move from one-off traces into repeatable grading once behavior stabilizes.